

FINAL COOPERATIVE EDUCATION PRESENTATION

# Development of an Enterprise Infrastructure Management System (EIMS)

Ratthabhumi Peansukmanee | Student ID: 66011198  
Computer Engineering (International Program), Faculty of Engineering  
King Mongkut's Institute of Technology Ladkrabang (KMITL)

Host Organization: KANEMATSU ELECTRONICS (THAILAND)  
Workplace Supervisor: Mr. Chalernpol Phuangchalam | Academic Supervisor: Asst. Prof. Dr. Phongsak Keeratiwintakorn

# Cooperative Education Context & Operational Responsibilities

## Host Enterprise Context

- **Host Organization:** KANEMATSU ELECTRONICS (THAILAND)
- **Industry Domain:** Enterprise Systems Integration, Compute Infrastructure, and Cloud Managed Services
- **Placement Duration:** Full-time 4th-year Cooperative Education (Academic Year 2026)
- **Workplace Division:** Enterprise Infrastructure Deployment & Systems Support Engineering
- **Supervision:** Under direct technical mentorship of Mr. Chalernpol Phuangchalam

## Routine Responsibilities vs. EIMS Capstone

### Routine Enterprise Engineering Duties

1. Bare-metal server staging, OS installation, and hardware component testing.
2. Network switch configuration, VLAN tagging, and firewall policy routing.
3. Ad-hoc endpoint troubleshooting and local diagnostic extraction on client servers.

### EIMS Capstone Development Scope

1. Engineered an authoritative central asset registry with hardware fingerprinting.
2. Designed an air-gapped, portable USB evidence collection utility for isolated servers.
3. Built an automated ingestion, deduplication, and AI-assisted investigation pipeline.

# Problem Statement: Infrastructure Evidence Fragmentation

**1. Network Enclave Isolation:** High-security production servers reside in air-gapped or segmented enclaves. Continuous streaming daemons cannot cross firewall boundaries, leaving endpoints invisible to central monitoring.

**2. Fragile Ephemeral Identity:** Dynamic IPs, DHCP leases, and server renames break historical tracking. Infrastructure lacks immutable hardware-anchored identity across server migrations.

**3. Labor-Intensive Manual Diagnostics:** Engineers must physically access servers and manually execute disparate PowerShell/CMD scripts, producing fragmented, unstructured notes without verification guarantees.

**4. High-Volume Telemetry Overload:** Windows endpoints generate thousands of routine event logs daily. Triage is slow, prone to oversight, and lacks deterministic prioritization.

**5. Absence of Provenance & History:** Ad-hoc investigation findings are rarely linked back to specific events, preventing retrospective auditing and historical incident cross-referencing.

# Project Objectives (Mapped 1:1 to Implementation & Evaluation)

**OBJ-1: Centralized Asset Registry:** Establish authoritative compute asset entity records anchored by cryptographic hardware composite fingerprints.

**OBJ-2: Portable Disconnected Collection:** Implement an episodic, read-only collection tool on portable media requiring zero target host software dependencies.

**OBJ-3: Structured Offline Evidence Persistence:** Extract hardware inventory and Windows Event Logs (System & Application, 24h window) into schema-validated JSON.

**OBJ-4: Deduplication & Prioritization Pipeline:** Enforce application-level sequential SHA-256 deduplication and deterministic severity ranking upon ingestion.

**OBJ-5: Bounded AI-assisted Investigation:** Deploy semantic embeddings and RAG triage bounded to at most 10 prioritized events per batch import to assist human engineers.

**OBJ-6: Historical Provenance & Auditability:** Maintain persistent AnalysisHistory records linking diagnostic findings, timestamps, and model provenance to the asset.

**OBJ-7: Centralized Investigation Dashboard:** Deliver a responsive web UI displaying endpoint profiles, compliance status, event logs, and AI findings.

# EIMS System Overview & Technology Stack

**Target Endpoints (Windows Server 2016-2025 / Windows 10-11):** Non-destructive read-only WMI/CIM queries and Windows Event Log `.evtx` channels.

**Portable USB Auditor (Embedded CPython 3.14.3 amd64):** Zero target dependencies. Self-contained runtime executing `Run-EIMS-Audit.bat` to output JSON.

**Core Backend API (FastAPI + Pydantic + Starlette):** Asynchronous Python 3.10+ gateway delivering OpenAPI 3.1 endpoints and strict schema validation.

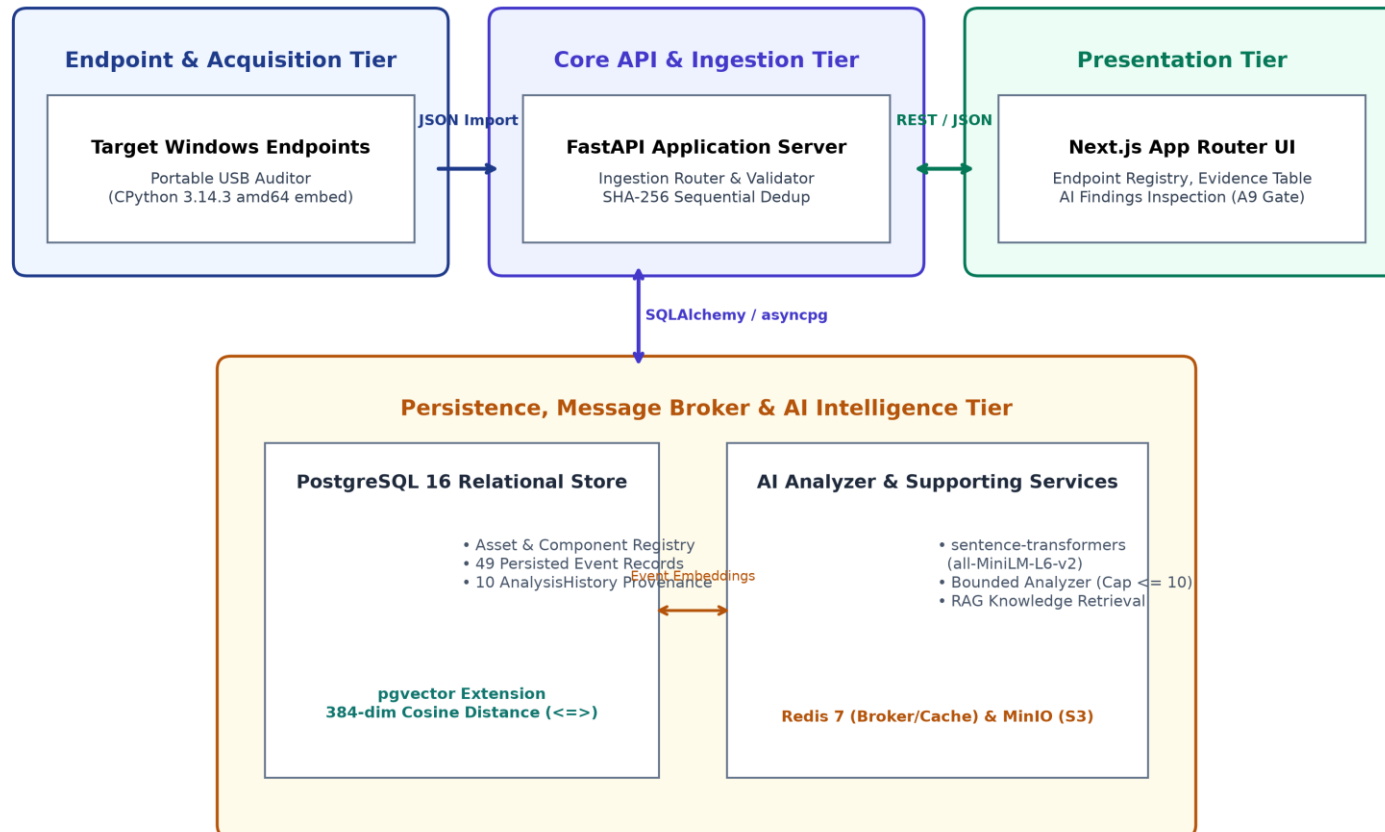
**Relational & Vector Store (PostgreSQL 16 + pgvector):** ACID transactional relational entity store combined with 384-dimensional vector cosine distance search.

**Broker & Caching (Redis 7 & MinIO Object Storage):** In-memory caching and message queuing paired with S3-compatible immutable blob persistence.

**AI Analysis Engine (sentence-transformers (all-MiniLM-L6-v2)):** Bounded retrieval-augmented generation (RAG) generating contextual diagnostic summaries.

**Operator Dashboard (Next.js 14 App Router + Tailwind CSS):** Server/Client hybrid rendering dashboard presenting real-time asset telemetry and AI findings.

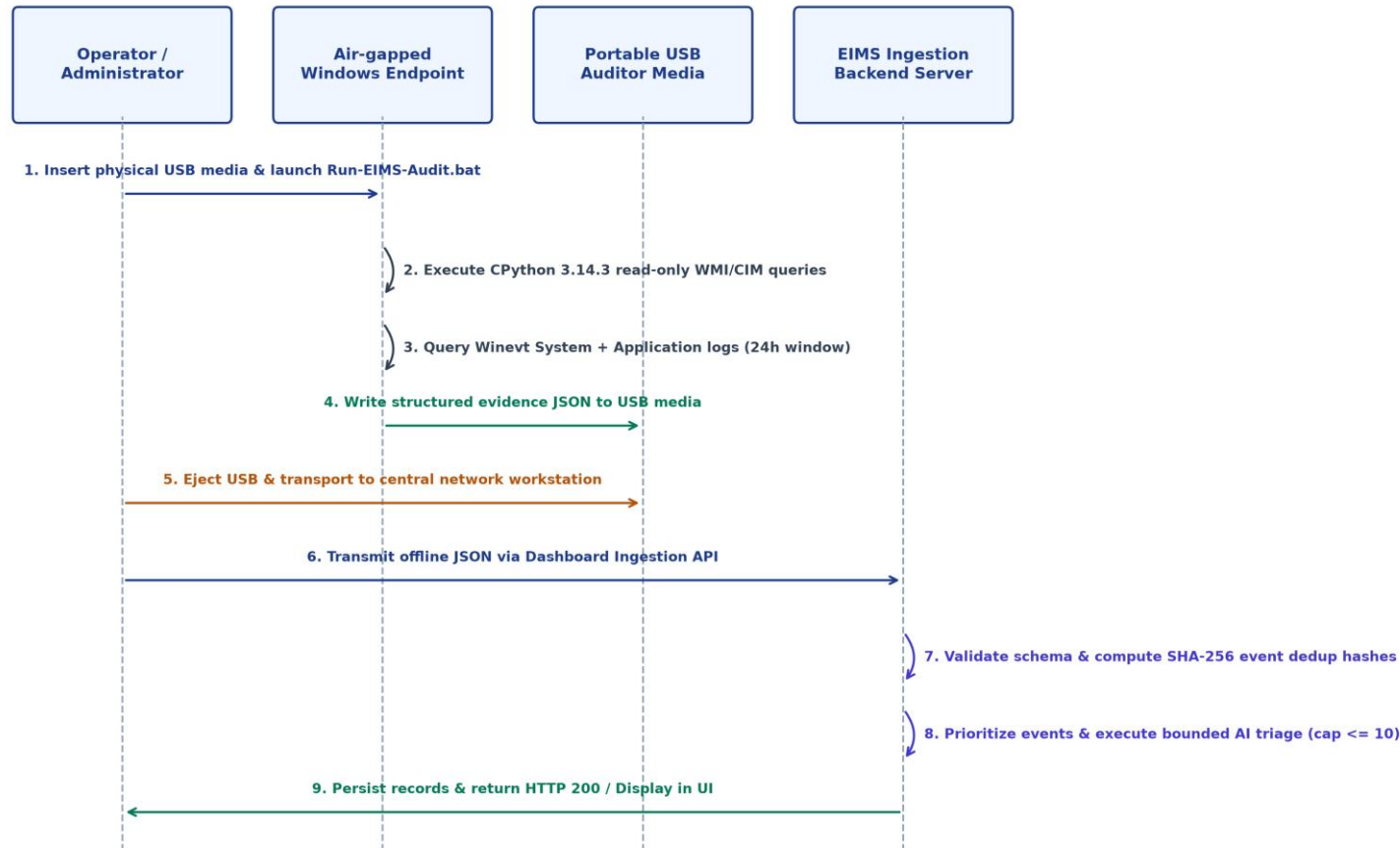
# Multi-Tier System Architecture (Hybrid Modular Monolith)



## Architectural Highlights

- **Hybrid Modular Monolith**: Consolidates domain services into a cohesive, maintainable codebase without microservice networking overhead.
- **Episodic Disconnected Collection**: USB Auditor bridges network isolation gaps without requiring continuous agent daemons.
- **Bounded Ingestion Pipeline**: Strict schema checks and sequential deduplication ensure database integrity.
- **Persistence & Vector Separation**: Relational tables store entities; pgvector handles mathematical distance operations.

# Portable Offline Evidence Collection & Ingestion Sequence



## Collection Guarantees

- **Zero Target Footprint:**

Requires no installed Python, Docker, or agent services on monitored servers.

- **Read-Only Assurance:**

Interrogates WMI/CIM and event logs with non-destructive, read-only queries.

- **Scoped Event Window:**

Captures System and Application logs spanning the preceding 24 hours (up to 500 events).

- **Defensible JSON Artifact:**

Generates human-readable, schema-validated JSON on the USB media prior to ingestion.

# Event Ingestion & Application-Level Deduplication

## Step 1: Validation

Pydantic payload validation rejecting malformed schemas (HTTP 422).

## Step 2: Asset Lookup

Resolve asset by cryptographic hardware composite fingerprint.

## Step 3: SHA-256 Dedup

Compute deterministic hash per event and filter duplicates.

## Step 4: Persistence

Persist novel event records to PostgreSQL relational table.

## Deduplication Hash Formulation

```
DedupKey = SHA-256(asset_id || channel || provider ||  
record_id || occurrence_time)
```

Because `record\_id` is monotonically assigned per channel by Windows and `occurrence\_time` is locked to kernel event creation, this hash is deterministic across multiple collections of the same physical event.

## Engineering Nuance: Sequential vs. Concurrent

### Sequential Application-Level Deduplication:

- The service queries existing keys and filters duplicates in memory prior to database insertion.
- Effectively prevents duplicate records during standard, sequential report imports.
- LIMITATION: Does NOT provide database-level concurrency guarantees. If two identical reports are submitted simultaneously across concurrent threads, race conditions can occur.

# Bounded AI-assisted Investigation & Knowledge Provenance

## Severity Prioritization & Bounded Cap

### Deterministic Severity Ranking:

Critical (Level 1) > Error (Level 2) > Warning (Level 3) > Information (Level 4)

### Workload Bounding (Analyzer Cap $\leq 10$ ):

- AI inference incurs latency and token overhead.
- The analyzer evaluates at most 10 prioritized events per batch import.
- High-impact failures are triaged first, preventing cognitive overload and pipeline backpressure.

## RAG Pipeline & Provenance Records

### Semantic Embeddings & Playbook Retrieval:

- Model: `all-MiniLM-L6-v2` (384-dimensional dense vectors).
- Metric: Cosine Distance via pgvector ( $\leq$  operator).
- Retrieval: Matches event logs to verified administrative playbooks.

### AnalysisHistory & Assistive Role:

- Persists diagnostic summary, severity score, and recommended action.
- Referential integrity: Links directly to asset entity.
- Assistive Aid: Accelerates human triage; does NOT act as autonomous root-cause authority or auto-remediator.

# Empirical Validation Strategy & Multi-Tier Gates

**Layer 1: Unit & Ingestion Tests:** Sprint 13 test suite executing 37 automated tests against real PostgreSQL instances validating schema parsing and deduplication.

**Layer 2: Frontend Compilation:** Strict TypeScript verification (`tsc --noEmit`) and Next.js production build (`next build`) verifying zero type mismatches.

**Layer 3: Physical USB Execution:** Executed portable package on target Windows Server hardware verifying read-only WMI/CIM and event log extraction.

**Layer 4: Filtered API Verification:** Queried backend endpoints (`GET /api/v1/history?asset\_id=...`) verifying HTTP 200 and asset-scoped response payloads.

**Layer 5: Browser Runtime (A9 Gate):** End-to-end browser inspection validating concurrent rendering of 49 event evidence records and 10 AI findings.

# Empirical Results & Measured Field Metrics

37 / 0 / 4

Sprint 13 Test Suite

37 Passed, 0 Failed, 4 Warnings

PASS

Frontend Build

TypeScript & Next.js Production Build

49 Events

Persisted Evidence

34 System + 15 Application (24h)

10 Records

AnalysisHistory

Bounded AI Cap (10/10 Analyzed)

## Field Verification Telemetry Breakdown

- **Evaluated Host Entity:** Synthetic test identifier `ASSET-01` representing enterprise Windows Server environment.
- **Channel Distribution:** 34 System channel events (hardware controller, services, drivers) + 15 Application channel events.
- **Temporal Window:** Preceding 24 hours of operational activity extracted from `%SystemRoot%\System32\Winevt\Logs\`.
- **Filtered API Verification:** `GET /api/v1/history?asset\_id=...` returned HTTP 200 with exactly the 10 asset-linked triage records.
- **Browser Gate A9 Outcome:** PASS — Verified simultaneous visibility of Event Evidence table and AI Findings inspection card. Placeholder 'No AI findings yet' correctly not rendered.

# Downstream Live End-to-End Verification Trace

**ASSET-01**

Target Host

**49 Events**

Persisted in DB

**Bounded Analyzer**

Cap  $\leq$  10

**10 Records**

AnalysisHistory

**HTTP 200**

Filtered API

**A9 PASS**

UI Verification

## Critical Distinction: Historical Acquisition vs. Downstream Verification

- **Downstream Verification Scope:**

The latest on-site verification confirmed downstream data integrity: persisted database rows, AnalysisHistory referential linkage, asset-scoped API filtering, and live Next.js UI rendering.

- **Historical Collection Separation:**

Physical USB acquisition, elevated script execution, and raw JSON extraction were rigorously proven in earlier field trials and were intentionally NOT re-run during the final downstream gate.

- **Auditability & Defense:**

By maintaining this clear boundary, the report avoids false claims of full re-execution while demonstrating that persisted evidence correctly propagates to operator interfaces.

# Engineering Challenges & Problem-Solving Lessons

## Sidebar Hydration Mismatch

**Problem:** Next.js SSR generated HTML differing from client browser window dimensions, triggering hydration warnings.

**Solution:** Refactored navigation to two-pass mounting using `useEffect` and CSS container queries.

**Lesson:** Strict separation of server-rendered layouts from dynamic client-state DOM components.

## Filtered History API HTTP 500

**Problem:** Querying `/api/v1/history?asset\_id=...` crashed when ORM attempted synchronous relationship loading.

**Solution:** Corrected asynchronous SQLAlchemy joins and applied explicit foreign-key filtering.

**Lesson:** Asynchronous database drivers require explicit eager-loading strategies to avoid event loop blocking.

## Environment-Specific Dataset Variance

**Problem:** Local development mocks diverged from actual Windows Server event schema formats.

**Solution:** Standardized on schema-validated Pydantic contract tests against real `.evtx` samples.

**Lesson:** Ingestion pipelines must validate data contracts at the gateway boundary rather than in persistence logic.

## Benchmark Data-Loss Prevention

**Problem:** Risk of destructive operations on production servers during automated evaluation.

**Solution:** Codified read-only collection rules, elevated UAC safety checks, and strict pre-flight assertions.

**Lesson:** Enterprise monitoring tools must adhere to non-destructive principles to be accepted in production.

# Scoped Limitations & Future Engineering Roadmap

## Acknowledged Scoped Limitations

- Windows-First Focus: Portable collection utility currently targets Windows WMI/winevt subsystems; Linux/macOS collectors deferred.
- System & Application Scope: Scoped strictly to System and Application channels; Security channel excluded from portable graduation scope.
- Sequential Dedup Only: Enforces application-level sequential protection; does not provide distributed concurrency guarantees.
- Bounded Triage Capacity: Evaluates at most 10 prioritized events per batch import.
- Assistive Role: AI suggests diagnoses; does not act as autonomous authority or apply automated patches.
- Academic Baseline: Prototype implementation; not a hardened commercial SIEM or certified forensic suite.

## Future Engineering Roadmap (ROADMAP-aligned)

- Cross-Endpoint Event Correlation: Implement temporal clustering to detect infrastructure-wide cascading failures.
- Database-Level Concurrency Protection: Introduce composite database constraints to guarantee race-condition safety under parallel imports.
- High-Availability Clustering: Deploy stateless backend pods and PostgreSQL replicas via Kubernetes.
- Enterprise Identity Federation: Integrate SAML 2.0 / OpenID Connect for Active Directory Single Sign-On (SSO).
- Multi-Platform Collectors: Extend embedded Python auditor to Linux systemd journals and macOS Unified Logging.

## CONCLUSION & CONTRIBUTIONS

# Summary of Project Achievements

1. Centralized Infrastructure Governance: Unified asset tracking with immutable hardware composite fingerprints.
2. Air-Gapped Evidence Acquisition: Proved non-destructive offline collection using embedded CPython 3.14.3 runtimes.
3. Defensible Deduplication & Prioritization: Implemented sequential SHA-256 event fingerprinting and deterministic triage.
4. Bounded AI Assistance: Delivered RAG-assisted investigation linked to auditable AnalysisHistory records.
5. Validated Downstream Pipeline: Confirmed end-to-end data integrity across 49 field events, HTTP 200 API, and Gate A9.

Thank you for your attention. I am now pleased to welcome your questions.